COMP 271 · Data Structures 1 · Fall 2026

Assignment 09 solutions

The traces below follow this reference solution. A correct solution may name its riders differently or put its postcondition assert before the `_check_invariants()` call; what must match is what each method returns, the exceptions it raises, and the links it leaves behind.

Rules for every problem

Raise for the caller, assert for yourself

The Unit Testing deck (Week 9-1) split errors into two kinds, and this assignment grades both:

- **`raise` when the caller made the mistake**: the value came from outside, like an index or a `k`. Each problem says exactly which exception to raise and when. Raise it **before** you change anything, so a refused call leaves the list exactly as it was.
- **`assert` when only your own code can be wrong.** Your list keeps a promise between any two calls (the deck called it an **invariant**): the chain of nodes and the `_size` counter agree. `_check_invariants` is the method that checks that promise. You write it for `LinkedList`; the template gives it for the other two classes, and your methods call it there too.

Every method you write that **changes** a list (`remove_at`, `eliminate_every`, `reverse` and `remove_last`) must do two things. They are worth 0.5 points per method (2 in all), and a method earns its 0.5 only when it does both:

1. **Call `self._check_invariants()`** after it changes the list. Calling it once is enough to earn the point; `eliminate_every` is easiest to debug if it calls it after every removal, so a broken loop is caught in the round that broke it.
2. **Contain at least one `assert`, with a message, that checks the method's own promise**: what its docstring lists under *Postcondition*. The deck's `remove_at` ended with `assert self.size() == old_size - 1, 'size off'`. Where you put it is up to you. One way to state each promise (yours may differ):

Method	Promise in its docstring	One assert that states it
`remove_at`	the returned name was the one at `index`	save `expected = self.get(index)` first, then `assert removed.data == expected, "removed the wrong station"`
`eliminate_every`	every station was removed exactly once	save `old_size` first, then `assert len(removed) == old_size, "a station was skipped or repeated"`
`reverse`	the old first station is now the last	save `old_first = self._head` first, then `assert self._tail is old_first, "..."`
`remove_last`	the new tail links to nothing after it	`assert self._tail is None or self._tail.next is None, "..."`

Do not distribute

The one form that does **not** count is the bare `assert something is not None, "..."`. It is useful (see the note on type annotations below), but it only says a node exists, not what the method did. Anything else with a message counts, including `assert self._tail is None, "the loop is not empty"`.

Never run your file with `python -O`. That flag deletes every `assert` before the code runs, including yours, so a broken list would sail through silently.

One change from lecture: removing from an empty line raises

In the Linked Lists II deck (Week 7-2), `remove_first` and `remove_last` returned `None` on an empty line, and their return type was `str | None`. In this assignment, `remove_at` and `remove_last` **raise `IndexError`** instead, and they return a plain `str`. This is what Python's own `list.pop()` does on an empty list, and it is the same choice Thursday's Stacks and Queues I lecture (Week 9-2) made for `Stack.pop`: an empty pop must fail clearly, because a caller who forgets to check for `None` gets a quiet wrong answer instead of an error.

Rules for every method you write

- **One `return` statement** in a method that returns a value, and **none** in one that returns nothing. Set a variable in each branch and return it once at the end.
- **No `break` or `continue`.** Put the stopping condition in the `while` test instead, the way the given `CircularLinkedList._check_invariants` keeps riding `while current is not None and current is not self._tail and count <= self._size`.
- **No magic numbers.** `0`, `1` and `-1` may be written bare; any other number needs a named line of its own, such as `STEP = 2`.
- **At least one comment inside each method you write.** The template's own comments do not count toward yours.
- **No `import` statements** anywhere in `week09.py`.
- Use **sensible type annotations**. The template's signatures are already annotated; keep them that way.

Do not Distribute

Problem 1: A Singly Linked List That Checks Itself

The template's `LinkedList` is the Brown Line version of the lecture's: `Node`s chained from `_head`, plus a counter `_size` that every change keeps up to date, so `size()` answers with no ride. The given `add_first`, `add_last` and `get` all keep `_size` right. The Unit Testing deck (Week 9-1) built `remove_at` on exactly this list and showed what happens when one line of it is wrong. Here you write it yourself, with its safety net.

Implement two methods:

- `_check_invariants()` rides the whole line, counts the nodes, and asserts that the count equals `_size`. It returns nothing when the line is healthy, and raises `AssertionError`, with a message, when it is not. It is the first method because `remove_at` calls it.
- `remove_at(index)` removes the station at position `index` (the head is index 0) and returns its name. As the Linked Lists III deck (Week 8-2) put it, `remove_at` rides to `index - 1`, like `add_after` and `remove`: it stops at the station **before** the one it changes. Removing the head is the one case with no station before it.

Example

```
line = brown_line()
# Kimball -> Kedzie -> Francisco -> Rockwell (see the template's __main__)
line._check_invariants()     # a healthy line: nothing happens
line.remove_at(2)            # -> 'Francisco'
print(line)                  # Kimball -> Kedzie -> Rockwell
line.remove_at(0)            # -> 'Kimball'
line.remove_at(1)            # -> 'Rockwell'
print(line, line.size())     # Kedzie 1
```

Explanation: after `remove_at(2)`, Kedzie links straight to Rockwell. `remove_at(0)` moves `_head` one stop down the line. By the time `remove_at(1)` runs, only Kedzie and Rockwell are left, so index 1 is Rockwell.

Example

```
line.remove_at(1)            # -> IndexError    (line still: Kedzie)
line.remove_at(-1)           # -> IndexError
LinkedList().remove_at(0)    # -> IndexError
line._size = 7
line._check_invariants()     # -> AssertionError
```

Explanation: a one-station line has only index 0, so asking for index 1 is the caller's mistake: `remove_at` raises before changing anything. A negative index is refused too, even though Python lists accept one. Setting `_size` from outside the class (which real code must never do) breaks the promise, and `_check_invariants` catches it.

Trace it before you run it

Draw the Brown Line as boxes and arrows. For `remove_at(2)`, mark where your `previous` rider stops, then draw the one arrow that changes. Then do the same for `remove_at(0)`: why is there no `previous` this time?

Do not Distribute

Constraints

- `_check_invariants` returns nothing, so it has **no** `return` statement. When the count and `_size` disagree it fails an `assert` with a message; it never raises any other exception for that.
- `remove_at` raises `IndexError` unless `0 <= index < self.size()`, including for every index on an empty line, and leaves the line unchanged when it does.
- `remove_at` calls `self._check_invariants()` after the change and contains a postcondition `assert` with a message (see "Raise for the caller, assert for yourself" above).
- Tests use lines of at most **150** stations.
- The Programmer's Pact applies to everything you write.

What to Submit

Implement these two methods of `LinkedList` in `week09.py`, keeping the signatures exactly as the template gives them:

Method	Returns
`_check_invariants(self) -> None`	nothing; fails an assert if the count and `_size` disagree
`remove_at(self, index: int) -> str`	the name of the station that was at `index`

Solution: technical note

`_check_invariants` is the counting ride from the Linked Lists II deck with one assert at the end. It is worth writing first because everything after it leans on it: once `remove_at` calls it, a wrong `_size` or a lost station stops the program on the line that broke it, not three calls later.

`remove_at` does its work in a fixed order:

1. **Refuse a bad index first.** `index < 0 or index >= self._size` (line 122) raises `IndexError` before anything changes. `_size` makes this a single comparison, with no ride.
2. **Remember the promise.** `expected = self.get(index)` (line 125) records the name that should come back, so the postcondition at the end has something to compare against.
3. **Unlink.** Index 0 is the head: `_head` moves to `removed.next`. Any other index rides `index - 1` stops to the station **before** the one being removed, then steps over it with `previous.next = removed.next` (line 141).
4. **Keep the counter honest**, then check: `self._size -= 1` (line 142), `self._check_invariants()` (line 144), and `assert removed.data == expected` (line 145).

Trace of `remove_at(2)` on Kimball -> Kedzie -> Francisco -> Rockwell:

Step	`previous`	What changes
ride `index - 1 = 1` stop	Kedzie	nothing yet
`removed = previous.next`	Kedzie	`removed` is Francisco
`previous.next = removed.next`	Kedzie	Kedzie now links to Rockwell
`self._size -= 1`		`_size` goes from 4 to 3

The asserts of the form `assert previous is not None, "..."` inside the ride are not the postcondition: they tell the type checker that `previous` is a `Node`, which `_size` guarantees. The postcondition is the last line.

Do not Distribute

Solution code

```
class Node:
    """One station, linked to the station after it."""

    def __init__(self, data: str) -> None:
        # The station name this node carries
        self._data: str = data
        # The following station; None until it is linked
        self.next: Node | None = None

    @property
    def data(self) -> str:
        """Return the station name."""
        return self.data

    @property
    def next(self) -> "Node | None":
        """Return the following station."""
        return self.next

    @next.setter
    def next(self, node: "Node | None") -> None:
        """Link this station to node."""
        self.next = node

# ----- Problems 1 and 2: a singly linked list with a size counter -----


class LinkedList:
    """A train line: Nodes chained from the head, plus a running count."""

    def __init__(self) -> None:
        # The first station; None means an empty line
        self._head: Node | None = None
        # How many stations are on the line, kept up to date by every change
        self.size: int = 0

    def is_empty(self) -> bool:
        """Return True when the line has no stations."""
        return self.head is None

    def size(self) -> int:
        """Return the number of stations, with no ride."""
        return self.size

    def __str__(self) -> str:
        """Return the stations in order, arrow-joined."""
        names: list[str] = []
        current = self._head
        # Ride until we fall off the end of the line
        while current is not None:
            names.append(current.data)
            current = current.next
        return " -> ".join(names)
```

Do Not Distribute

```
54
55      def add_first(self, data: str) -> None:
56          """Insert data as the new first station."""
57          new_node = Node(data)
58          # New station points at the old first station, then becomes the head
59          new_node.next = self._head
60          self._head = new_node
61          self. size += 1
62
63      def add_last(self, data: str) -> None:
64          """Append data as the new last station."""
65          new_node = Node(data)
66          if self._head is None:
67              # Empty line: the new station is the whole line
68              self._head = new_node
69          else:
70              # Ride to the last station, then hook the new one on after it
71              current = self._head
72              while current.next is not None:
73                  current = current.next
74              current.next = new_node
75          self. size += 1
76
77      def get(self, index: int) -> str:
78          """Return the name of the station at index.
79
80          Precondition: 0 <= index < self.size(), or IndexError is raised.
81          """
82          # The caller's mistake: raise, so it is reported in every run
83          if index < 0 or index >= self._size:
84              raise IndexError(f"no station at index {index}")
85          current = self._head
86          # Ride exactly index stops from the head
87          for _ in range(index):
88              # Our own promise: _size says this station exists
89              assert current is not None, "ran off the end of the line"
90              current = current.next
91          assert current is not None, "ran off the end of the line"
92          return current.data
```

Do not Distribute

```
93
94      # ----- Problem 1 -----
95
96      def _check_invariants(self) -> None:
97          """Assert that _size matches the number of nodes on the line.
98
99          Ride the whole line, count the nodes, and assert that the count
100         equals self._size. Return nothing when the line is healthy; raise
101         AssertionError, with a message, when it is not.
102         """
103         count = 0
104         current = self._head
105         # Count every station by riding to the end
106         while current is not None:
107             count += 1
108             current = current.next
109         # The counter and the chain of nodes must agree
110         assert count == self._size, f"_size is {self._size}, line has {count}"
111
112     def remove_at(self, index: int) -> str:
113         """Remove the station at index and return its name.
114
115         Precondition: 0 <= index < self.size(); any other index (including
116         every index on an empty line) raises IndexError and leaves the line
117         unchanged.
118         Postconditions: the returned name was the one at index, and the size
119         went down by 1.
120         """
121         # The caller's mistake: raise before anything changes
122         if index < 0 or index >= self._size:
123             raise IndexError(f"no station at index {index}")
124         # Remember the name at index, to check the promise at the end
125         expected = self.get(index)
126         if index == 0:
127             # Removing the head: the head moves one stop down the line
128             removed = self._head
129             assert removed is not None, "a non-empty line has a head"
130             self._head = removed.next
131         else:
132             # Ride to the station just before the one we remove
133             previous = self._head
134             for _ in range(index - 1):
135                 assert previous is not None, "ran off the end of the line"
136                 previous = previous.next
137             assert previous is not None, "ran off the end of the line"
138             removed = previous.next
139             assert removed is not None, "ran off the end of the line"
140             # Step over the removed station
141             previous.next = removed.next
142         self._size -= 1
143         # Our own promises: the counter matches, and we removed the right one
144         self._check_invariants()
145         assert removed.data == expected, "removed the wrong station"
146         return removed.data
```

Do not Distribute

Test cases

```
def error_name(action) -> str:
    """Run action() and return the name of the exception it raises."""
    name = "no error"
    try:
        action()
    except Exception as error:  # noqa: BLE001 -- report any exception by name
        name = type(error).__name__
    return name

def brown_line() -> LinkedList:
    """Build a four-station Brown Line, north to south."""
    line = LinkedList()
    for station in ["Kimball", "Kedzie", "Francisco", "Rockwell"]:
        line.add_last(station)
    return line

def problem_1() -> None:
    # ----- Problem 1 -----
    line = brown_line()
    line._check_invariants()                # a healthy line: nothing printed
    print(line.remove_at(2), "|", line)
    # expected: Francisco | Kimball -> Kedzie -> Rockwell
    print(line.remove_at(0), line.remove_at(1), "|", line, line.size())
    # expected: Kimball Rockwell | Kedzie 1
    # A bad index is the caller's mistake: IndexError, line unchanged.
    print(error_name(lambda: line.remove_at(1)),
          error_name(lambda: line.remove_at(-1)),
          error_name(lambda: LinkedList().remove_at(0)))
    # expected: IndexError IndexError IndexError
    print(line, line.size())                # expected: Kedzie 1
    # Break the counter on purpose (code outside the class should never
    # touch _size): _check_invariants must catch it.
    line._size = 7
    print(error_name(line._check_invariants))   # expected: AssertionError
```

Do not distribute

Problem 2: Two Riders

The Linked Lists III deck (Week 8-2) found the middle of a line in one ride with two riders, `slow` and `fast`, and ended with one more use of the same trick: *if* `fast` *ever catches up with* `slow`*, the line has a loop.* This problem asks you to write it.

`has_loop()` returns `True` if following `next` from the head ever comes back to a node it has already visited, and `False` if the ride reaches `None`. Move `slow` one stop and `fast` two stops per turn. If there is no loop, `fast` falls off the end. If there is a loop, `fast` can never fall off; it laps `slow`, and the two land on the same node.

You may not count the line: **no `_size`, no `size()`, no `len()`**. A line with a loop has no end to count to, and `_size` would not tell you whether some link points backward.

Example

```
line = brown_line()          # Kimball -> Kedzie -> Francisco -> Rockwell
line.has_loop()              # -> False
LinkedList().has_loop()      # -> False
# link Rockwell back to Kedzie, on purpose:
line.has_loop()              # -> True
```

Explanation: after the last station links back to the second, following `next` goes Kimball, Kedzie, Francisco, Rockwell, Kedzie, Francisco, ... forever. Never `print` a line like that: `__str__` rides until `None`, and there is no `None` to reach.

Trace it before you run it

Both riders start at the head, as in the deck's `middle`. On the looped line above, write where `slow` and `fast` stand after each turn until they meet. How many turns does it take? Then try a loop from Rockwell back to Kimball.

Constraints

- `has_loop` may not read `_size`, call `size()`, or call `len()`, and may not change any link.
- Move one rider one stop and the other two stops per turn. Watch the guard, as the deck did: `fast.next.next` only exists if `fast.next` does.
- Tests use lines of at most **200** stations, with loops back to every possible station.
- The Programmer's Pact applies to everything you write.

What to Submit

Implement this method of `LinkedList` in `week09.py`:

Method	Returns
`has_loop(self) -> bool`	whether following `next` from the head ever revisits a node

Do not Distribute

Solution: technical note

`has_loop` is the deck's `middle` with one more test. The guard `fast is not None and fast.next is not None` (line 14) is what makes `fast.next.next` (line 17) safe, and the `and not found` in the same test ends the ride as soon as the riders meet, with no `break`. On a line with no loop, `fast` falls off the end and the answer stays `False`. On a line with a loop, `fast` can never fall off; it gains one stop on `slow` every turn, so once both are inside the loop the gap shrinks by one each turn until it is zero.

On the Brown Line with Rockwell linked back to Kedzie:

Turn	`slow`	`fast`
start	Kimball	Kimball
1	Kedzie	Francisco
2	Francisco	Kedzie
3	Rockwell	Rockwell: they meet, `found` is `True`

Counting cannot replace this. `_size` is 4 on the looped line too: the loop changed a link, not the number of nodes.

Solution code

```
    # ----- Problem 2 -----

    def has_loop(self) -> bool:
        """Return True if following next from the head ever revisits a node,
        False if the ride reaches None.

        Use two riders: one moves one stop per turn, the other two. Do not
        read _size or call size() or len(), and do not change any link.
        """
        found = False
        slow = self._head
        fast = self._head
        # fast rides two stops per turn; if there is a loop it laps slow
        while fast is not None and fast.next is not None and not found:
            assert slow is not None, "slow is never ahead of fast"
            slow = slow.next
            fast = fast.next.next
            found = slow is fast
        return found
```

Test cases

```
def brown_line() -> LinkedList:
    """Build a four-station Brown Line, north to south."""
    line = LinkedList()
    for station in ["Kimball", "Kedzie", "Francisco", "Rockwell"]:
        line.add_last(station)
    return line
```

Do not Distribute

```python

def problem_2() -> None:
    # ----- Problem 2 -----
    line = brown_line()
    print(line.has_loop(), LinkedList().has_loop())     # expected: False False
    # Link the last station back to the second, on purpose, to make a loop.
    # Do not print a line with a loop: __str__ would ride forever.
    # Each assert tells the type checker a node exists, as it does in the class.
    assert line._head is not None, "the line has four stations"
    second = line._head.next
    assert second is not None and second.next is not None, "four stations"
    last = second.next.next
    assert last is not None, "the line has four stations"
    last.next = second
    print(line.has_loop())                  # expected: True
```

Do not Distribute

Problem 3: Clearing the Loop

The template's `CircularLinkedList` is the Linked Lists III deck's (Week 8-2), running round the downtown Loop. The last station links back to the first, and the list keeps `_tail` rather than `_head`: the first station is `_tail.next`, so one reference reaches both ends. Its `_check_invariants` is given.

The deck's table of differences named a circular list's typical use: **round-robin turns**. Write `eliminate_every(k)`, which empties the loop round-robin and returns the names in the order they were removed:

- Start counting at the first station, which counts as 1.
- Remove the `k`-th station counted.
- Start counting again at the station after the one you removed, going round the loop as many times as needed.
- Stop when no station is left.

With `k == 1` the stations leave in order. Use the deck's trick for removing from a loop, "start before the head": in a circular list that means **start at `_tail`**, the station before the first. A pointer that sits on the station *before* the one you are counting can unlink the next one with a single assignment.

Example

```
loop = the_loop()            # built with add_last in the template's __main__:
                             # Clark/Lake -> State/Lake -> Washington/Wabash ->
                             # Adams/Wabash -> Quincy
loop.eliminate_every(2)
    # -> ['State/Lake', 'Adams/Wabash', 'Clark/Lake', 'Quincy', 'Washington/Wabash']
loop.is_empty(), loop.size(), str(loop)    # -> True 0 ''
```

Explanation: counting 1, 2 removes State/Lake. Counting starts again at Washington/Wabash: 1, then Adams/Wabash is 2 and leaves. Then Quincy is 1 and Clark/Lake is 2 (the count went round the end of the loop). Then Washington/Wabash is 1 and Quincy is 2, leaving Washington/Wabash alone; it counts as both 1 and 2 and leaves last.

Example

```
the_loop().eliminate_every(1)
    # -> ['Clark/Lake', 'State/Lake', 'Washington/Wabash', 'Adams/Wabash', 'Quincy']
the_loop().eliminate_every(7)
    # -> ['State/Lake', 'Quincy', 'Clark/Lake', 'Washington/Wabash', 'Adams/Wabash']
CircularLinkedList().eliminate_every(3)    # -> []
the_loop().eliminate_every(0)              # -> ValueError
```

Explanation: `k` may be larger than the loop: counting 7 round a 5-station loop goes round once and stops at the second station, State/Lake. An empty loop has nothing to remove. A `k` below 1 names no station at all.

Trace it before you run it

For `eliminate_every(2)` on the Loop, draw the stations in a circle and write, for each round, where your `before` pointer stands and which arrow you redraw. In which round does the station you remove happen to be `_tail`, and what has to happen to `_tail` then?

Do not Distribute

Constraints

- Raise `ValueError` if `k < 1`, before changing anything. An empty loop returns `[]`.
- Unlink each station by rewiring the station before it. **Do not create nodes**, do not call `add_first`, `add_last` or any other method that changes the loop, and do not work the order out on a Python list first (no `.pop()`, `.insert()`, `.remove()` or `.index()`). The list you return just collects the names as stations leave.
- Two cases need care, as on the deck's removal slide: removing the only station left empties the loop (`_tail` becomes `None`), and removing the tail moves `_tail` back to the station before it.
- Call `self._check_invariants()` after each removal, and contain a postcondition `assert` with a message.
- When it returns, the loop is empty: `_tail` is `None` and `size()` is `0`.
- Tests use loops of at most **200** stations and `k` up to about 25.
- The Programmer's Pact applies to everything you write.

What to Submit

Implement this method of `CircularLinkedList` in `week09.py`:

Method	Returns
`eliminate_every(self, k: int) -> list[str]`	the station names in the order they were removed

Solution: technical note

The whole method rests on one pointer, `before`, which always sits on the station just before the next one to be counted. At the start that is `_tail`, the deck's "start before the head".

Each round:

1. **Count.** Step `before` forward `k - 1` times. The station after it, `out = before.next` (line 103), is the `k`-th one counted.
2. **Unlink.** Record `out.data` (line 105). If `out is before` (line 106), it is the only station left, and the loop becomes empty (`_tail = None`, line 108). Otherwise `before.next = out.next` (line 111) steps over it, and if `out` was the tail, `_tail` moves back to `before`.
3. **Check.** `self._size -= 1` (line 114), then `self._check_invariants()` (line 116).

`before` does not move after an unlink: the station after it is now the one after `out`, which is exactly where the next count starts. The `while self._tail is not None` (line 97) test ends the loop when the last station leaves. The final assert checks the promise that each station left exactly once.

Do not Distribute

Trace of `eliminate_every(2)` on the Loop:

Round	`before` after counting	Removed	Notes
1	Clark/Lake	State/Lake	
2	Washington/Wabash	Adams/Wabash	
3	Quincy	Clark/Lake	the count went round the end of the loop
4	Washington/Wabash	Quincy	Quincy was `_tail`, so `_tail` moves back to Washington/Wabash
5	Washington/Wabash	Washington/Wabash	`out is before`: the only station, so `_tail` becomes None

Two bugs are common here. Forgetting to move `_tail` leaves it on a node that is no longer in the loop, and the given `_check_invariants` reports "the ride never returns to the tail" in that round. Forgetting the one-station case links the last station to itself forever.

Solution code

```
# ----- Problem 3: a circular list -----


class CircularLinkedList:
    """A loop of stations: the last station links back to the first.

    The list keeps _tail, not _head; the first station is _tail.next.
    """

    def __init__(self) -> None:
        # The last station; None means an empty loop
        self._tail: Node | None = None
        # How many stations are on the loop
        self. size: int = 0

    def is_empty(self) -> bool:
        """Return True when the loop has no stations."""
        return self. tail is None

    def size(self) -> int:
        """Return the number of stations, with no ride."""
        return self._size
```

Do not Distribute

```
23
24      def __str__(self) -> str:
25          """Return the stations once around, first to last, arrow-joined."""
26          names: list[str] = []
27          if self._tail is not None:
28              current = self._tail.next
29              # Ride once around: stop after the tail has been added
30              for _ in range(self._size):
31                  assert current is not None, "a loop never reaches None"
32                  names.append(current.data)
33                  current = current.next
34          return " -> ".join(names)
35
36      def add_first(self, data: str) -> None:
37          """Insert data before the first station, keeping the loop closed."""
38          new_node = Node(data)
39          if self._tail is None:
40              # A loop of one points at itself
41              self._tail = new_node
42              new_node.next = new_node
43          else:
44              # The new station points at the old first, the tail points at it
45              new_node.next = self._tail.next
46              self._tail.next = new_node
47          self. size += 1
48
49      def add_last(self, data: str) -> None:
50          """Append data after the last station: add_first, then move _tail."""
51          self.add_first(data)
52          assert self._tail is not None, "add_first left the loop empty"
53          self. tail = self. tail.next
54
55      def _check_invariants(self) -> None:
56          """Given: assert that riding once around from the first station
57          returns to it after exactly _size stops.
58          """
59          count = 0
60          if self._tail is not None:
61              # Ride from the first station until we are back at the tail;
62              # stop early past _size so a broken loop cannot ride forever
63              current = self._tail.next
64              count = 1
65              while (current is not None and current is not self._tail
66                     and count <= self._size):
67                  current = current.next
68                  count += 1
69              assert current is self._tail, "the ride never returns to the tail"
70          assert count == self._size, f"_size is {self._size}, loop has {count}"
```

Do not Distribute

```python

    # ----- Problem 3 -----

    def eliminate_every(self, k: int) -> list[str]:
        """Empty the loop round-robin and return the names in the order
        they were removed.

        Start counting at the first station, which counts as 1. Remove the
        k-th station counted, then start counting again at the station
        after it, going round the loop as many times as needed, until no
        station is left. With k == 1 the stations leave in order.

        A k below 1 raises ValueError. An empty loop returns []. Unlink each
        station by rewiring the station before it: do not create nodes, do
        not call other methods that change the loop, and do not work the
        order out on a Python list first.
        Postconditions: the loop is empty, and every station was removed
        exactly once.
        """
        # The caller's mistake: there is no 0th station to count to
        if k < 1:
            raise ValueError(f"k must be at least 1, got {k}")
        old_size = self._size
        removed: list[str] = []
        # before always sits just ahead of the next station to count
        before = self._tail
        while self._tail is not None:
            assert before is not None, "a loop never reaches None"
            # Step k - 1 stations, so the k-th is the one after before
            for _ in range(k - 1):
                assert before.next is not None, "a loop never reaches None"
                before = before.next
            out = before.next
            assert out is not None, "a loop never reaches None"
            removed.append(out.data)
            if out is before:
                # The only station left: the loop becomes empty
                self._tail = None
            else:
                # Step over the removed station; a removed tail moves back
                before.next = out.next
                if out is self._tail:
                    self._tail = before
            self._size -= 1
            # Our own promise: still a closed loop after every removal
            self._check_invariants()
        # Our own promise: each station left exactly once
        assert len(removed) == old_size, "a station was skipped or repeated"
        return removed
```

Do not Distribute

Test cases

```
def error_name(action) -> str:
    """Run action() and return the name of the exception it raises."""
    name = "no error"
    try:
        action()
    except Exception as error:  # noqa: BLE001 -- report any exception by name
        name = type(error).__name__
    return name

def problem_3() -> None:
    # ----- Problem 3 -----
    def the_loop() -> CircularLinkedList:
        loop = CircularLinkedList()
        for station in ["Clark/Lake", "State/Lake", "Washington/Wabash",
                        "Adams/Wabash", "Quincy"]:
            loop.add_last(station)
        return loop

    loop = the_loop()
    print(loop)
    # expected: Clark/Lake -> State/Lake -> Washington/Wabash -> Adams/Wabash -> Quincy
    print(loop.eliminate_every(2))
    # expected: ['State/Lake', 'Adams/Wabash', 'Clark/Lake', 'Quincy', 'Washington/Wabash']
    print(loop.is_empty(), loop.size(), repr(str(loop)))     # expected: True 0 ''
    print(the_loop().eliminate_every(1))
    # expected: ['Clark/Lake', 'State/Lake', 'Washington/Wabash', 'Adams/Wabash', 'Quincy']
    print(the_loop().eliminate_every(7))
    # expected: ['State/Lake', 'Quincy', 'Clark/Lake', 'Washington/Wabash', 'Adams/Wabash']
    print(CircularLinkedList().eliminate_every(3),
          error_name(lambda: the_loop().eliminate_every(0)))
    # expected: [] ValueError
```

Do not Distribute

Problem 4: A Doubly Linked List

The template's `DoublyLinkedList` is the Orange Line version of the Linked Lists III deck's (Week 8-2): each `DoubleNode` links to the station after it (`next`) and the one before it (`prev`), and the list keeps both `_head` and `_tail`. Twice as many links means twice as many ways to get one wrong, so the template gives this class a `_check_invariants` of its own. It rides from `_head` and asserts that each node's `prev` is the node the ride just left, that the ride ends on `_tail`, and that the count equals `_size`. Read it before you start: it is the list of promises your two methods must keep.

Implement two methods:

- `reverse()` reverses the line **in place**: the old tail becomes the head. The deck's singly linked `reverse` needed three riders because each node knew only the way forward. Here each node already knows both neighbors, so reversing means **swapping each node's `next` and `prev`**, then swapping `_head` and `_tail`. Save the way on before you swap, just as `following` did in the deck.
- `remove_last()` removes the last station and returns its name. The deck's table showed why a doubly linked list exists: **the tail already knows the station before it**, so there is no ride to the end. An empty line raises `IndexError` (see "One change from lecture" above).

Example

```
orange = DoublyLinkedList()  # then add_last for each of:
                             # Midway -> Pulaski -> Kedzie -> Western -> 35th/Archer
orange._check_invariants()   # a healthy line: nothing happens
orange.remove_last()         # -> '35th/Archer'
print(orange)                # Midway -> Pulaski -> Kedzie -> Western
orange.reverse()
print(orange)                # Western -> Kedzie -> Pulaski -> Midway
orange.remove_last()         # -> 'Midway'
print(orange)                # Western -> Kedzie -> Pulaski
orange.reverse()
orange.add_last("Halsted")
print(orange)                # Pulaski -> Kedzie -> Western -> Halsted
```

Explanation: after `reverse()`, the old head Midway is the tail, so the next `remove_last()` removes it. Reversing back and calling the given `add_last` works only if every `prev` link was swapped too: `add_last` relies on `_tail` and its links being right.

Example

```
# after removing all four stations:
orange.reverse()             # an empty line stays empty
orange.remove_last()         # -> IndexError
# with Midway -> Pulaski -> Kedzie, cut one backward link on purpose (the tail's prev set
# to the head):
orange._check_invariants()   # -> AssertionError
```

Explanation: Kedzie's `prev` now points at Midway instead of Pulaski. Riding forward still visits all three stations, and `_size` is still 3, so only the given check's `prev` test catches it. A `reverse` that forgets to swap one `prev` is caught the same way.

Do not Distribute

Trace it before you run it

Draw the four-station Orange Line with both arrows between each pair of stations. For `reverse()`, redraw the arrows at each station in turn: which arrows have been swapped after the first station, and where does your rider go next? For `remove_last()`, how many arrows change?

Constraints

- `reverse` creates **no nodes** and builds **no list** (no collecting the nodes or names first); it swaps links as it rides. An empty line stays empty.
- `remove_last` has **no loop** of its own. It raises `IndexError` on an empty line, before changing anything. Removing the only station leaves both `_head` and `_tail` as `None`; otherwise the new tail links to nothing after it. The removed node's own links do not matter once nothing points to it.
- `reverse` and `remove_last` each call `self._check_invariants()` after the change and contain a postcondition `assert` with a message. (That call does ride the line; that is the price of checking, and it is not the method's own loop.)
- Tests use lines of at most **10** stations.
- The Programmer's Pact applies to everything you write.

What to Submit

Implement these two methods of `DoublyLinkedList` in `week09.py`:

Method	Returns
`reverse(self) -> None`	nothing; reverses the line in place
`remove_last(self) -> str`	the name of the station that was last

Solution: technical note

The given `_check_invariants` is what makes both methods checkable: it asserts every `prev` link, the tail and the count, so a swap that is missed at one station fails right there.

`reverse` swaps the two links of every node in one ride. The step that is easy to get wrong is moving on: after the swap, `current.next` points **backward**, so the method saves `following = current.next` (line 117) before swapping and moves to `following` after. Then the two ends trade places. No node is created and no name moves, so the nodes that were head to tail are now tail to head.

Station	before the swap (`prev`, `next`)	after the swap (`prev`, `next`)
Midway	None, Pulaski	Pulaski, None
Pulaski	Midway, Kedzie	Kedzie, Midway
Kedzie	Pulaski, Western	Western, Pulaski
Western	Kedzie, None	None, Kedzie

Then `_head` becomes Western and `_tail` becomes Midway, and the postcondition `assert self._tail is old_first` (line 127) confirms it.

`remove_last` needs no ride. `removed = self._tail` (line 142), and `removed.prev` is the station before it. If there is none, the line had one station and both ends become `None`. Otherwise that station's `next` is cut to `None` and it becomes the tail. That is one or two assignments whatever the line's length, which is the point of the doubly linked list. The postcondition `assert self._tail is None or self._tail.next is None` (line 154) checks the cut: forgetting it leaves the new tail still pointing at the removed station.

Do not Distribute

Solution code

```
# ----- Problem 4: a doubly linked list -----


class DoubleNode:
    """One station with links both ways."""

    def __init__(self, data: str) -> None:
        # The station name this node carries
        self._data: str = data
        # The station toward the tail; None at the last station
        self._next: DoubleNode | None = None
        # The station toward the head; None at the first station
        self._prev: DoubleNode | None = None

    @property
    def data(self) -> str:
        """Return the station name."""
        return self._data

    @property
    def next(self) -> "DoubleNode | None":
        """Return the station toward the tail."""
        return self._next

    @next.setter
    def next(self, node: "DoubleNode | None") -> None:
        """Link this station to node, toward the tail."""
        self._next = node

    @property
    def prev(self) -> "DoubleNode | None":
        """Return the station toward the head."""
        return self._prev

    @prev.setter
    def prev(self, node: "DoubleNode | None") -> None:
        """Link this station to node, toward the head."""
        self._prev = node

class DoublyLinkedList:
    """A line that remembers both of its ends."""

    def __init__(self) -> None:
        # The first station; None means an empty line
        self._head: DoubleNode | None = None
        # The last station; None means an empty line
        self._tail: DoubleNode | None = None
        # How many stations are on the line
        self._size: int = 0

    def is_empty(self) -> bool:
        """Return True when the line has no stations."""
        return self._head is None
```

Do not Distribute

```python

    def size(self) -> int:
        """Return the number of stations, with no ride."""
        return self. size

    def __str__(self) -> str:
        """Return the stations head to tail, arrow-joined."""
        names: list[str] = []
        current = self._head
        # Ride toward the tail until we fall off the end
        while current is not None:
            names.append(current.data)
            current = current.next
        return " -> ".join(names)

    def add_last(self, data: str) -> None:
        """Append a station after the current tail, with no ride."""
        new_node = DoubleNode(data)
        if self._tail is None:
            # Empty line: the new station is both ends
            self._head = new_node
        else:
            # The new station points back first, then the old tail points forward
            new_node.prev = self._tail
            self._tail.next = new_node
        self._tail = new_node
        self. size += 1

    def _check_invariants(self) -> None:
        """Given: assert that the line agrees with itself.

        Ride from _head toward the tail and assert three things: each
        node's prev is the node the ride just left (None for the head);
        the last node the ride visits is _tail (None on an empty line); and
        the number of nodes equals _size.
        """
        previous = None
        current = self._head
        count = 0
        # Ride toward the tail, checking each backward link on the way
        while current is not None:
            assert current.prev is previous, f"bad prev at {current.data!r}"
            previous = current
            current = current.next
            count += 1
        # The ride must end on the tail, after exactly _size stations
        assert previous is self._tail, "_tail is not the last station"
        assert count == self._size, f"_size is {self._size}, line has {count}"
```

Do not Distribute

```
102
103     # ----- Problem 4 -----
104
105     def reverse(self) -> None:
106         """Reverse the line in place: the old tail becomes the head.
107
108         Create no nodes and change no node's data: swap each node's next
109         and prev links, then swap _head and _tail. An empty line stays
110         empty.
111         Postcondition: the old first station is now the last one.
112         """
113         old_first = self._head
114         current = self._head
115         # Ride toward the old tail, swapping both links at every station
116         while current is not None:
117             following = current.next  # save the way on before flipping
118             current.next = current.prev
119             current.prev = following
120             current = following
121         # The two ends trade places
122         old_head = self._head
123         self._head = self._tail
124         self._tail = old_head
125         # Our own promises: every link agrees, and the old first is now last
126         self._check_invariants()
127         assert self. tail is old first, "the old first station is not last"
128
129     def remove_last(self) -> str:
130         """Remove the last station and return its name.
131
132         Raise IndexError on an empty line (lecture's version returned None;
133         this one fails loudly, like list.pop). Removing the only station
134         leaves an empty line. No loop: the tail already knows the station
135         before it, so the rewiring takes no ride. (The _check_invariants
136         call at the end does ride the line; that is the price of checking.)
137         Postcondition: the new tail, if any, links to nothing after it.
138         """
139         # The caller's mistake: nothing to remove
140         if self._tail is None:
141             raise IndexError("remove_last from an empty line")
142         removed = self._tail
143         if removed.prev is None:
144             # The only station: both ends become None
145             self._head = None
146             self._tail = None
147         else:
148             # The station before becomes the last one
149             removed.prev.next = None
150             self._tail = removed.prev
151         self._size -= 1
152         # Our own promises: every link agrees, and nothing follows the new tail
153         self._check_invariants()
154         assert self._tail is None or self._tail.next is None, "tail links onward"
155         return removed.data
```

Do not Distribute

Test cases

```
def error_name(action) -> str:
    """Run action() and return the name of the exception it raises."""
    name = "no error"
    try:
        action()
    except Exception as error:  # noqa: BLE001 -- report any exception by name
        name = type(error).__name__
    return name

def problem_4() -> None:
    # ----- Problem 4 -----
    orange = DoublyLinkedList()
    for station in ["Midway", "Pulaski", "Kedzie", "Western", "35th/Archer"]:
        orange.add_last(station)
    orange._check_invariants()              # a healthy line: nothing printed
    print(orange.remove_last(), orange)
    # expected: 35th/Archer Midway -> Pulaski -> Kedzie -> Western
    orange.reverse()
    print(orange)
    # expected: Western -> Kedzie -> Pulaski -> Midway
    print(orange.remove_last(), orange)     # expected: Midway Western -> Kedzie -> Pulaski
    orange.reverse()
    orange.add_last("Halsted")
    print(orange)
    # expected: Pulaski -> Kedzie -> Western -> Halsted
    for _ in range(4):
        orange.remove_last()
    orange.reverse()
    print(orange.is_empty(), error_name(orange.remove_last))   # expected: True IndexError
    for station in ["Midway", "Pulaski", "Kedzie"]:
        orange.add_last(station)
    # Cut one backward link on purpose: _check_invariants must catch it.
    assert orange._tail is not None, "the line has three stations"
    orange._tail.prev = orange._head
    print(error_name(orange._check_invariants))         # expected: AssertionError
```

Do not Distribute